

UE5 C++ MASTERY ÇALIŞMA MERKEZİ · SUJET

02

OBSTACLE ASSAULT

Sıfırdan hareket eden, döner ve engelli bir platformer kurarak C++'ın mutlak temellerini — değişkenler, fonksiyonlar, sınıflar, Tick döngüsü — inşa etmek.

SÜRE

4 saat 35 dakika

DERS SAYISI

34 ders

ALİŞTİRMA

8 zorunlu + 1 bonus

XP

120 (+30 bonus)

BÖLÜM 0 — GENEL BİLGİLER VE KURALLAR

0.1 Önsöz

Bu bölümde artık kod yazıyorsun. İlk C++ Actor'ünü oluşturup onu hareket ettireceksin — basit görünen bu görev aslında bir oyun motorunun kalbindeki üç kavramı (zaman, konum, karar verme) bir araya getiriyor. Alıştırmalar bilerek küçük adımlara bölündü: her biri bir öncekinin üzerine, tek bir yeni fikir ekleyerek inşa ediyor.

0.2 Amaç

Struct, üye değişken, fonksiyon ve Tick/DeltaTime kavramlarını kullanarak, hedefe doğru hareket eden, overshoot yapmayan, gidip gelen ve döner bir platform sistemini C++'ta sıfırdan yazmak; bunu bir Blueprint Child Class üzerinden oynanabilir bir seviyeye dönüştürmek.

0.3 Genel Yönlendirmeler

- Her alıştırmayı bitirdiğinde derle ve Play-in-Editor ile test et — derlenmeyen kod bir sonraki adıma taşınmaz.
- UE_LOG'u bir hata ayıklama aracı olarak erken ve sık kullan; “neden çalışmıyor” demeden önce değerleri logla.
- Kod tekrarını fark ettiğinde (ör. hem SetActorLocation hem SetActorRotation için benzer mantık) bunu bir fonksiyona çıkar.
- Sayısal sabitleri (hız, mesafe) UPROPERTY olarak dışa açıp Blueprint'ten ayarlanabilir hale getir.

0.4 Değerlendirme Kriterleri (Kendi Kendine Savunma)

Bir 42 öğrencisinin projesini savunması gibi, aşağıdaki sorulara videoya veya nota bakmadan, kendi cümlelerinle cevap verebilmelisin:

- Tick fonksiyonu ne zaman, hangi sıklıkla çağrılır ve DeltaTime tam olarak neyi temsil eder?
- Overshoot problemi nedir ve senin çözümün neden çalışıyor?
- Bir struct ile ayrı ayrı üç float değişken arasındaki fark nedir? Neden struct'ı tercih ettin?
- C++ sınıfından türetilen Blueprint Child Class'ta hangi değerler C++'ta, hangileri Blueprint'te kalmalı, neden?

0.5 Zorunlu ve Yasak

✓ ZORUNLU

- Her hareket hesabında DeltaTime kullanmak (frame-rate bağımsızlık)
- Overshoot durumunu kontrol edip düzeltmek
- Sayısal parametreleri (hız, mesafe) UPROPERTY ile Blueprint'e açmak

✗ YASAK

- Tick içinde DeltaTime kullanmadan sabit bir sayıyla hareket ettirmek
- Aynı hareket mantığını kopyala-yapıştır ile birden fazla yere yazmak
- Hedefe SetActorLocation ile “ışınlayarak” overshoot'u “çözmüş gibi yapmak”

0.6 XP ve Rank Sistemi

Bu bölümün her alıştırması bir XP değeri taşır. Zorunlu kısmı eksiksiz bitirince **120 XP** kazanırsın; bonus dahil üst sınır **150 XP**'dir (bonus, ancak zorunlu %100 tamamlandıktan sonra açılır). Kazandığın XP'yi Notion'daki “XP Takip Sistemi” veritabanında işaretleyerek genel kurs rankını görebilirsin — bu bölümü bonusuyla bitirince **Junior Gameplay Programcısı** rankına yaklaşırsın.

BÖLÜM I — ZORUNLU KISIM

Alıştırmalar birbiri üzerine inşa olacak şekilde sıralanmıştır — bir öncekini bitirmeden sonrakine geçme. Her alıştırmaya, ilgili Notion sayfasındaki “Konu Anlatımı” bölümünün karşılık gelen kısmına dayanır.

EXERCISE 00 · 10 XP

İlk Actor ve Log

İlgili dersler: Creating a C++ Class · Variables and Data Types · Logging Variables

Amaç — AActor'dan türeyen ilk C++ sınıfını oluşturup BeginPlay içinde bir mesajı Output Log'a yazdırmak.

► Yapılacaklar

- Yeni bir C++ Actor sınıfı oluştur (ör. AMovingPlatform).
- BeginPlay fonksiyonunu override et.
- İçinde UE_LOG ile kendi ismini ve bir sayısal değeri ekrana yazdır.
- Sınıftan sahneye bir örnek (instance) sürükleyin, Play'e bas.

⚠ Kısıtlar

- BeginPlay yerine yanlışlıkla constructor'a mantık yazma — ikisi farklı zamanlarda çalışır.

✓ Beklenen Sonuç

- Play'e basınca Output Log'da beklenen mesaj bir kez görünür.

EXERCISE 01 · 10 XP

Hedef Struct'ı

İlgili dersler: Member Variables · Structs

Amaç — Platformun başlangıç ve hedef konumunu tek bir mantıksal birimde tutan bir struct tanımlamak.

► Yapılacaklar

- Başlangıç ve hedef konumu (FVector) tutan bir USTRUCT tanımla.
- Bu struct'tan bir üye değişken ekle sınıfına.
- BeginPlay'de başlangıç konumunu GetActorLocation ile doldur.

⚠ Kısıtlar

- Konumları iki ayrı bağımsız FVector değişkeni olarak tutma — struct'ı kullan.

✓ Beklenen Sonuç

- Struct içindeki iki alan da BeginPlay sonrası doğru değerlerle dolu.

EXERCISE 02 · 20 XP

Hareketli Platform

İlgili dersler: SetActorLocation · Tick · GetActorLocation · Delta Time · Velocity

Amaç — Platformu, Tick içinde DeltaTime kullanarak hedef konuma doğru sabit hızla hareket ettirmek.

► Yapılacaklar

- Tick fonksiyonunu override et.
- Mevcut konum ile hedef arasındaki yön vektörünü hesapla, normalize et.
- Bunu bir hız değeriyle ve DeltaTime ile çarpıp SetActorLocation ile uygula.

⚠ Kısıtlar

- Hızı DeltaTime ile çarpmadan kullanma — farklı FPS'lerde farklı hız üretir.

✓ Beklenen Sonuç

- Platform, farklı frame hızlarında (ör. Editor'de FPS sınırını değiştirerek) aynı sürede hedefe ulaşıyor.

EXERCISE 03 · 20 XP

Overshoot Düzeltmesi

İlgili dersler: Overshoot Problem · Correcting Overshoot · Function Return Values

Amaç — Platformun hedefi “aşmasını” önleyen bir kontrol fonksiyonu yazmak.

► Yapılacaklar

- Kalan mesafeyi hesaplayan, float döndüren bir fonksiyon yaz.
- Bu mesafe, o frame'de alınacak adımdan küçükse hareketi hedefte durdur.
- If Statement ile bu durumu kontrol et.

⚠ Kısıtlar

- Overshoot kontrolünü sadece “gözle test edip iyi görünüyor” diye atlama — uç durumları (çok yüksek hız) dene.

✓ Beklenen Sonuç

- Hız değeri çok yüksek ayarlansa bile platform hedefte duruyor, titremeden.

EXERCISE 04 · 15 XP

Gidiş-Dönüş Mekanığı

İlgili ders: If Statements (pekiştirme)

Amaç — Platform hedefe ulaştığında yönünü tersine çevirip başlangıç noktasına geri dönmesini sağlamak.

► Yapılacaklar

- Bir bool bayrak (ör. bShouldReturn) ekle.
- Hedefe ulaşıldığında bu bayrağı tersine çevir.
- Tick içindeki hedef konumu bu bayrağa göre seç (başlangıç ya da hedef).

⚠ Kısıtlar

- Bayrağı her frame gereksiz yere tersine çevirme — sadece hedefe varıldığı anda.

✓ Beklenen Sonuç

- Platform sürekli başlangıç ile hedef arasında gidip geliyor.

EXERCISE 05 · 15 XP

Döner Platform

İlgili ders: Rotating Platforms

Amaç — Konum hareketiyle aynı mantığı (Tick + DeltaTime) rotasyona uygulayan ikinci bir platform türü yazmak.

► Yapılacaklar

- Yeni bir Actor sınıfı (ya da aynı sınıfa opsiyonel bir mod) oluştur.
- Tick içinde DeltaTime'a bağlı sabit bir açısal hızla AddActorLocalRotation çağır.

⚠ Kısıtlar

- Rotasyon hızını da DeltaTime'dan bağımsız sabit derece olarak yazma.

✓ Beklenen Sonuç

- Platform, FPS'ten bağımsız olarak sabit açısal hızda dönüyor.

EXERCISE 06 · 15 XP

Blueprint Child Class

İlgili dersler: Classes And Instances · Blueprint Child Classes

Amaç — C++ sınıfından bir Blueprint Child Class türetip görsel mesh/materyal atamalarını orada yapmak.

► Yapılacaklar

- C++ sınıfından bir Blueprint Child Class oluşturun.
- Static Mesh Component ekleyin, bir mesh ve materyal ata.
- Hız/mesafe gibi sayısal parametreleri Details panelinden değiştirip test et.

⚠ Kısıtlar

- Mesh/materyal atamasını C++ constructor'ına hard-code etme — Blueprint'te kalmalı.

✓ Beklenen Sonuç

- Aynı C++ sınıfından, farklı görsellere sahip birden fazla Blueprint türetilebiliyor.

EXERCISE 07 · 15 XP

Seviye ve Paketleme

İlgili dersler: Level Design · Packaging The Game

Amaç — Yazılan platform türlerini kullanarak oynanabilir bir seviye kurmak ve oyunu bağımsız bir yürütülebilir dosya olarak paketlemek.

► Yapılacaklar

- En az 3 farklı platform türünü (düz, döner, gidip-gelen) seviyeye yerleştir.
- Karakterin hareket parametrelerini (zıplama, hız) Blueprint'ten ince ayar yap.
- Oyunu Windows/Mac için paketle ve paketlenen sürümü Editor dışında çalıştır.

⚠ Kısıtlar

- Paketlemeden önce Editor'de en az bir baştan-sona test oynayıp yapmadan paketleme.

✓ Beklenen Sonuç

- Paketlenmiş .exe/.app, Editor açık olmadan bağımsız çalışıyor.

BÖLÜM II — BONUS (İSTEĞE BAĞLI)

Bonus, zorunlu kısım tam ve eksiksiz çalışır durumda olmadan başlanmamalıdır — tıpkı 42'deki mantık gibi: önce sağlam temel, sonra süsleme.

BONUS · +30 XP (zorunlu %100 tamamlanınca açılır)

Waypoint Listesi ile Çoklu Duraklı Platform

Bağımsız pratik — TArray'i önden tanıma (Dungeon Escape'e köprü)

Amaç — Tek bir hedef yerine bir TArray içinde tutulan birden fazla waypoint arasında sırayla giden bir platform yazmak.

► Yapılacaklar

- FVector dizisi tutan bir TArray üye değişkeni ekle.
- Platform bir waypoint'e ulaştığında diziye bakıp bir sonrakine geçsin.
- Son waypoint'e ulaşınca başa dönsün (dairesel) ya da geri sarsın (senin tercihin).

⚠ Kısıtlar

- Dizi boşken (0 eleman) Tick'in çökmeden davranmasını sağla.

✓ Beklenen Sonuç

- Platform, 3 veya daha fazla noktayı sırayla ziyaret ediyor.

TESLİM VE ÖZ-DEĞERLENDİRME

Bu bölümü bitirdiğinde elinde çalışan, paketlenabilir küçük bir oyun olmalı. Devam etmeden önce 0.4'teki soruları kontrol et; özellikle Delta Time ve overshoot konusunda tereddüdün varsa Dungeon Escape'e geçmeden bu sujet'i tekrar gözden geçir — bir sonraki bölüm bu temeller üzerine pointer ve component kavramlarını ekleyecek.